

Day 1 Foundations & First Deployment

Participant Manual

Platform	AxisPay (fictional)
Kubernetes	v1.36
Environment	Ubuntu 26.04 LTS · Minikube
Built	14 August 2026

Every merchant, customer, card token, acquirer and transaction in this manual is fictional. No real institution is represented and no card number exists anywhere in this platform — only tokens. The platform is **not** PCI-DSS compliant and is not presented as such; it uses PCI-shaped constraints to make security controls consequential in a training context.

Day 2 — Reliability, Resource Governance and Controlled Change

AxisPay · Kubernetes Comprehensive · Participant Manual, Chapter 2

What changed today

Yesterday you built a platform. Today you made it trustworthy.

Yesterday	Today
Pods land wherever the scheduler guesses	Every workload declares what it needs
Running is the only health signal	Three probes, separated by consequence
Capacity is whatever you typed	Autoscaling on real CPU load
Only Deployments	Deployment, DaemonSet, Job, CronJob
A deploy drops payments	Zero-downtime release, measured

Chapter structure is the same 16-point template as Chapter 1.

2.1 Resource requests and limits

1. What it is

Two numbers per container. **Requests** tell the scheduler how much to reserve. **Limits** tell the kernel what ceiling to enforce at runtime. They feed two different systems that never talk to each other.

2. Why it exists

Without requests, the scheduler has no idea what a pod needs and packs nodes until they collapse. Without limits, one leaking process takes the whole node down — including every unrelated workload on it.

3. The business problem

AxisPay's p99 authorisation is **217 ms** against a **300 ms** SLO. That is 83 ms of headroom.

A merchant reports intermittent slowness. Nothing is down. Nothing has restarted. The logs are clean. The cause is a CPU limit set by someone who wanted to be careful, and it is consuming the entire 83 ms — with no log line, no event and no restart.

4. How it works

SCHEDULING		RUNTIME	
uses requests only		uses limits only	
node allocatable	2000m	CPU over limit	-> THROTTLED
already reserved	1400m	MEM over limit	-> OOMKILLED (137)
pod requests	300m -> fits		
	-> 300m RESERVED		

The reservation is not a measurement. A node that is 5% busy but 95% *reserved* will refuse new pods. This surprises people the first time they see an idle node reject work.

5. Internal architecture

The kubelet translates requests and limits into Linux cgroup v2 settings:

Field	cgroup control	Effect
<code>requests.cpu</code>	<code>cpu.weight</code>	Relative share when the CPU is contended
<code>limits.cpu</code>	<code>cpu.max</code> (quota/period)	Hard throttle — the process is descheduled for the rest of the period
<code>requests.memory</code>	(scheduling only)	Not enforced at runtime
<code>limits.memory</code>	<code>memory.max</code>	Exceed it and the kernel OOM-killer terminates the process

CPU quota is enforced per **100 ms period**. A container limited to 500m gets 50 ms of CPU per 100 ms. Burn it in 20 ms and you are frozen for the remaining 80 ms — which is where sawtooth latency comes from.

6. Component interactions

```
you           declare requests + limits
scheduler     filters nodes by unreserved capacity (requests)
kubelet       writes cgroup values (limits)
kernel        throttles CPU / OOM-kills on memory
metrics-server scrapes actual usage
HPA           usage ÷ REQUEST = utilisation
```

Requests appear twice: once as the scheduler's input, once as the HPA's denominator. That is why one wrong number breaks two systems.

7. Enterprise example

A payments platform runs a quarterly "right-sizing" review: p95 usage over 30 days becomes the new request, p99 plus 100% becomes the limit. Requests that are too high are as expensive as limits that are too low — over-requesting silently makes cluster capacity unusable for everyone else, and it does not appear in your own metrics at all.

8. Real-world analogy

A restaurant booking. The **request** is the table you reserve — held for you whether or not you turn up. The **limit** is the maximum party size that table will take. Reserve a table for two and try to seat six, and you are turned away.

Where it breaks: a restaurant cannot seat a walk-in at your reserved table. Kubernetes *can* let another pod burst into CPU you reserved but are not using. Reserved CPU is not idle CPU.

9. Best practices

Practice	Reason
Measure before you set	<code>kubectl top</code> under load, several samples, not one idle reading
<code>request = steady state + ~30%</code>	The scheduler must reserve for a normal day
<code>limit = peak + ~100%</code>	Room for a genuine spike without eating the node
Always set a memory limit	Without one, a leak takes down the node, not just the pod
Be cautious with CPU limits	Throttling is invisible. Many teams set memory limits and omit CPU limits deliberately.
Never set <code>limit < request</code>	Nonsensical: the scheduler reserves more than the kernel allows. Always a defect.
Re-measure after every significant release	v1.1.0 added two HTTP client pools to <code>payment-service</code>

10. Common mistakes

Mistake	Symptom
No requests at all	BestEffort QoS, evicted first, HPA reports <code><unknown></code>
CPU limit too low	Latency breach with no log, no event, no restart
Memory limit too low	<code>OOMKilled</code> , exit 137, CrashLoopBackOff
<code>limit below request</code>	Pod schedules then dies immediately — this is INC-2
Requests set far too high	Cluster capacity silently wasted; invisible in your own metrics
Setting limits == requests everywhere	Guaranteed QoS, zero burst headroom, throttled at exactly the wrong moment
Sizing from one idle sample	Correct at 03:00, throttled at 09:00

11. Security considerations

Threat	Control	Residual risk
One tenant exhausts a node (noisy neighbour / DoS)	Limits on every container; ResourceQuota per namespace	A pod can still saturate network or disk I/O
Memory-exhaustion attack on a node	Memory limits + eviction thresholds	Kernel-level exhaustion still possible
Pod with no limits admitted	LimitRange <code>default</code>	Only applies to pods created after it
Resource-based side channels between tenants	Node isolation for sensitive workloads	Shared kernel remains shared

12. Performance considerations

- **CPU throttling is invisible.** The only signals are latency and `nr_throttled / throttled_usec` in `/sys/fs/cgroup/cpu.stat`.
- **The 100 ms period matters.** A latency-sensitive service can be badly throttled while showing modest average CPU, because the burst is concentrated.
- **Memory has no throttle.** There is no gentle degradation — only OOMKill.
- **Over-requesting costs real money.** Reserved-but-unused capacity cannot be scheduled by anyone.
- **Leading indicator:** `container_cpu_cfs_throttled_periods_total / container_cpu_cfs_periods_total` above ~5% means the limit is too low.

13. High availability

Resources are a per-pod concern, but they determine cluster-level survivability. Requests that are too low let the scheduler overcommit a node; when reality catches up, the kubelet evicts — **BestEffort first, then Burstable, Guaranteed last**. Accurate requests are what make eviction ordering meaningful.

14. Disaster recovery

Resource settings are configuration and recover with the manifest. The failure worth planning for is **node pressure**: if a node runs out of memory, the kubelet evicts pods by QoS class and by how far each exceeds its request. Workloads that must survive should have accurate requests and, for the truly critical, a `PriorityClass` (Day 4).

15. Monitoring

Metric	Threshold
<code>container_cpu_cfs_throttled_periods_total</code> ratio	> 5% of periods
<code>container_memory_working_set_bytes</code> / <code>limit</code>	> 85%
<code>kube_pod_container_status_last_terminated_reason{reason="OOMKilled"}</code>	Any — page
<code>kube_pod_container_resource_requests</code> vs actual usage	Persistent gap = over-requesting
<code>kube_resourcequota</code> used vs hard	> 80%

16. Troubleshooting

Symptom	Cause	Command	Fix
Slow, no errors, no restarts	CPU throttling	<code>kubectl exec ... -- cat /sys/fs/cgroup/cpu.stat</code>	Raise the CPU limit
OOMKilled, exit 137	Memory limit too low, or a leak	<code>kubectl describe pod</code> → Last State	Raise the limit; check for a leak
Pending, "Insufficient cpu"	No node has enough unreserved	<code>kubectl describe pod</code> ; <code>kubectl describe node</code>	Lower requests or add capacity
exceeded quota	Namespace budget full	<code>kubectl describe quota -n <ns></code>	Lower requests or raise the quota
Pod dies seconds after starting, clean logs	<code>limit</code> below what the process needs	<code>kubectl describe pod</code> → Last State	Raise the limit — this is INC-2
Node evicting pods	Node under memory pressure	<code>kubectl describe node</code> → Conditions	Fix requests; add capacity

Interview questions

- Which does the scheduler use — requests or limits?** *Requests only. Limits are enforced by the kernel at runtime and are invisible to scheduling.*
- What happens when a container exceeds its CPU limit? Its memory limit? CPU:** *throttled — slowed down, no error, no event, no log. Memory: OOMKilled, exit 137, restarted. CPU is compressible, memory is not.*
- What are the three QoS classes and how are they assigned?** *Guaranteed (requests == limits for every container and both resources), Burstable (requests set, limits higher or absent), BestEffort (nothing declared). Eviction order is BestEffort, Burstable, Guaranteed.*
- Why might you deliberately NOT set a CPU limit?** *(senior) Because throttling is invisible and often worse than the problem it prevents. Requests already guarantee a share under contention. Many teams set memory limits (to contain leaks) and omit CPU*

limits (to allow bursting), accepting noisy-neighbour risk in exchange for predictable latency. It is a real trade with real advocates on both sides.

5. **A pod has `requests.memory: 96Mi` and `limits.memory: 48Mi`. What happens and what should have caught it?** *(senior) The scheduler reserves 96Mi; the kernel kills the process above 48Mi. It schedules and then dies immediately, repeatedly. A validating admission policy or a CI policy check should reject any manifest where a limit is below its request — it is never correct.*
-

2.2 ResourceQuota and LimitRange

1. What it is

Namespace-level governance. A **ResourceQuota** caps the total a namespace may consume. A **LimitRange** supplies per-container defaults and enforces minimum, maximum and ratio bounds.

2. Why it exists

Per-pod resources are the developer's concern. Namespace ceilings are the platform team's. Without them, any namespace can consume the whole cluster.

3. The business problem

A misconfigured CI pipeline created 400 pods in a staging namespace on a shared cluster. It exhausted the node pool, and **production payment pods could not be scheduled for eleven minutes**. No malice, no application bug — just no ceiling.

4. How it works

Both are **admission-time** controls. They run when an object is created and reject it if it does not fit. Neither affects anything already running.

```
Deployment submitted
-> LimitRange    fills in missing requests/limits, checks min/max/ratio
-> ResourceQuota is there room in the namespace budget?
-> admitted, or REJECTED (the pod is never created)
```

5. Internal architecture

The dependency between them catches everyone. A ResourceQuota on `requests.cpu` makes any pod *without* a CPU request unschedulable — Kubernetes cannot count what was never declared. The LimitRange's `defaultRequest` supplies the missing number.

A ResourceQuota without a LimitRange breaks every manifest that forgot to set resources.

AxisPay's quota on `axispay-core` :

Resource	Hard	Sized for
<code>requests.cpu</code>	2	1920m at full HPA scale
<code>requests.memory</code>	2Gi	1520Mi at full HPA scale
<code>limits.cpu</code>	9	8700m at full HPA scale
<code>limits.memory</code>	5Gi	3904Mi at full HPA scale
<code>Pods</code>	40	~19 at peak

Requests must fit real capacity. Limits may be oversubscribed. The scheduler reserves requests, so their sum can never exceed what the cluster has. Limits are a per-container ceiling; it is normal and correct for their sum to exceed capacity — that is what Burstable QoS is for.

But the quota must still allow the HPA to reach `maxReplicas`, or autoscaling stops partway up with `FailedCreate` on the ReplicaSet — during exactly the spike it exists to absorb.

6. Component interactions

The rejection surfaces on the **ReplicaSet**, not the Deployment. The Deployment reports `0/1` and looks fine. `kubectl describe rs` is where the `FailedCreate` event lives. This is the most common way a quota rejection gets misdiagnosed.

7. Enterprise example

A bank automates namespace creation: requesting `payments-prod` provisions a ResourceQuota sized to the team's budget, a LimitRange with sensible defaults, a default-deny NetworkPolicy, RBAC bound to the owning group, and Pod Security labels. Nobody creates a namespace by hand, so nobody forgets a control.

8. Real-world analogy

A departmental budget with a per-item spending cap. The ResourceQuota is the annual budget; the LimitRange is "no single purchase over R50,000, and nothing under R100 needs approval". You can be refused for exceeding either.

9. Best practices

Practice	Reason
Never a quota without a LimitRange	Otherwise every under-declared manifest is rejected
Size the quota for HPA max, not steady state	Or autoscaling fails during the spike
Quota object counts, not just compute	Stops a runaway controller creating 10,000 objects
Set <code>maxLimitRequestRatio</code>	Stops workloads lying to the scheduler
Automate namespace creation with its controls attached	Manual creation always forgets something
Alert at 80% quota utilisation	Before rejections start, not after

10. Common mistakes

Mistake	Symptom
Quota without LimitRange	Every bare manifest rejected; confusing for developers
Quota sized for steady state	HPA silently stops mid-scale during a spike
Only checking <code>kubectl get deploy</code>	The rejection is on the ReplicaSet; you never see it
Forgetting object-count quotas	A CI loop fills etcd with Jobs
Quota on limits but not requests	The scheduler can still overcommit the nodes

11. Security considerations

Threat	Control	Residual risk
Resource-exhaustion DoS by one tenant	ResourceQuota per namespace	Network and disk I/O are not quota'd
Object-count flooding of etcd	<code>count/*</code> quotas	Cluster-scoped objects are not covered
Privileged over-provisioning	RBAC on quota objects themselves	Cluster-admin can always raise them

12. Performance considerations

Quota evaluation is a synchronous admission step and adds a small latency to every create. At very high object-creation rates the quota controller can become a bottleneck — visible as `apiserver_admission_controller_admission_duration_seconds` climbing for `ResourceQuota`.

13. High availability

Governance objects are metadata and inherit control-plane HA. Their contribution to availability is indirect but real: they prevent one namespace from making the cluster unschedulable for everyone else.

14. Disaster recovery

Trivially reproducible from manifests. The one operational care: raising a quota during an incident is a legitimate break-glass action, but it must be reflected back into Git or the next apply reverts it — quietly, at the worst possible time.

15. Monitoring

Metric	Threshold
<code>kube_resourcequota{type="used"} / {type="hard"}</code>	> 80%
FailedCreate events on ReplicaSets	Any
<code>apiserver_admission_controller_admission_duration_seconds{name="ResourceQuota"}</code>	Rising

16. Troubleshooting

Symptom	Cause	Command	Fix
exceeded quota	Budget full	<code>kubectl describe quota -n <ns></code>	Lower requests, scale down, or raise the quota
Deployment 0/1, no pod	Quota rejection on the ReplicaSet	<code>kubectl describe rs -n <ns></code>	Read the FailedCreate
Every new pod rejected	Quota with no LimitRange	<code>kubectl get limitrange -n <ns></code>	Add a LimitRange with <code>defaultRequest</code>
maximum cpu usage per Container	Exceeds LimitRange max	<code>kubectl describe limitrange</code>	Lower the limit
ratio ... is higher than max	request/limit gap too wide	Same	Raise the request or lower the limit
HPA stalls below max	Quota reached mid-scale	<code>describe hpa ; describe quota</code>	Size the quota for HPA max

Interview questions

- Why does a ResourceQuota need a LimitRange?** A quota on requests makes pods without requests unschedulable — the quota cannot count an undeclared value. The LimitRange's `defaultRequest` supplies it. Without one, a quota breaks every manifest that omitted resources.

2. **Where does a quota rejection appear?** *On the ReplicaSet, as a `FailedCreate` event. The Deployment just reports fewer replicas than desired. Checking only `kubect1 get deploy` hides the cause completely.*
 3. **Why can `limits` be oversubscribed but `requests` cannot?** *(senior) Requests are reservations the scheduler must honour, so their sum is bounded by real capacity. Limits are per-container ceilings enforced by the kernel; a pod may burst into headroom neighbours are not using. Oversubscribing limits is how Burstable QoS delivers value.*
 4. **How do you size a quota for a namespace with autoscaling?** *(senior) For `maxReplicas`, not steady state. A quota sized to normal load causes the HPA to stop scaling partway up, with `FailedCreate` on the ReplicaSet — during the exact traffic spike it exists to absorb, and with a symptom that looks nothing like the cause.*
-

2.3 Health probes

1. What it is

Three checks the kubelet runs against your container, each with a **different consequence** when it fails.

2. Why it exists

Without probes, the only health signal Kubernetes has is "did the process start?" — which is not the same as "can this pod take a payment?"

3. The business problem

Yesterday's incident took out **all three** `payment-service` replicas simultaneously. A bad image rolled out to every pod and nothing stopped it. The review asked: *why did the rollout continue after the first pod failed?*

Because Kubernetes had no way to know it had failed.

4. How it works

Probe	Question	Consequence of failure	Check dependencies?
startup	Has initialisation finished?	Keep waiting; liveness suspended	No
liveness	Is this process unrecoverable?	Container RESTARTED	Never
readiness	Can <i>this pod</i> serve right now?	Removed from endpoints. Not restarted.	Yes — that is the point

Learn the consequence, not the name. Every probe mistake in production is made by someone who knew the definitions.

AxisPay's timings:

Probe	period × threshold	Reacts in
startup	2 s × 30	60 s budget to start
liveness	10 s × 3	30 s before restart
readiness	5 s × 2	10 s before traffic stops

Readiness reacts three times faster than liveness, deliberately. Taking a pod out of rotation is cheap and reversible; restarting it is expensive and destroys in-flight work. When in doubt, stop sending traffic — do not restart.

5. Internal architecture

The kubelet runs every probe locally — it never goes through a Service. Probe results feed two paths:

- **Liveness failure** → kubelet restarts the container in place. The pod object, its IP and its node do not change; only `restartCount` increments.
- **Readiness failure** → kubelet updates pod `status.conditions`. The endpoint controller observes it and removes the address from the EndpointSlice. kube-proxy on every node reprograms.

Only `Ready` pods appear in a Service's endpoints. That is the direct link between a probe and traffic routing, and the mechanism behind zero-downtime deployment.

6. Component interactions

```
kubelet    -> probe fails
kubelet    -> writes pod status Ready=false
endpoint controller -> removes address from EndpointSlice
kube-proxy (every node) -> reprograms iptables/IPVS
                        ~10-15s total, including propagation
```

That propagation delay is real, and it is why `preStop` exists.

7. Enterprise example

A payments platform requires every service to expose `/healthz` (process only) and `/readyz` (dependency-aware) before it can be deployed. The rule is enforced by an admission policy. It exists because a single team once pointed liveness at a database health check and turned a 40-second failover into a 20-minute platform outage.

8. Real-world analogy

A shop assistant. **Liveness** asks "are you conscious?" — if not, send them home. **Readiness** asks "can you serve the next customer right now?" — if they are on the phone to a supplier, stop queueing customers at them, but do not fire them.

Firing every assistant because the supplier's phone line is busy is the cascading-failure bug.

9. Best practices

Practice	Reason
Liveness checks the process only	Anything else causes correlated restarts
Readiness checks critical dependencies	That is the entire point
Always add a startup probe to a slow starter	Otherwise liveness races initialisation
Readiness faster than liveness	Stopping traffic is cheaper than restarting
<code>successThreshold: 1</code> on readiness	Anything higher delays recovery for no benefit
Classify dependencies critical vs non-critical	Not everything should take a pod out of rotation
Keep probes cheap	The kubelet runs them constantly, per pod
Exclude probe paths from access logs	Otherwise they dominate log volume — see <code>SILENT_PATHS</code>

10. Common mistakes

Mistake	Symptom
Liveness checks a dependency	Every replica restarts together on a blip → total outage
No readiness probe	Rollout "succeeds" while dropping traffic
No startup probe on a slow starter	CrashLoopBackOff on a cold cache
Liveness too aggressive	Restarts under load, never recovers
Readiness <code>successThreshold > 1</code>	Pod never becomes ready
Probe healthier than reality	Probes pass, payments fail
Expensive probe	Probe traffic becomes a measurable load

11. Security considerations

Threat	Control	Residual risk
Probe endpoint leaks internal state	Return status only, never config or versions of dependencies	Timing differences can still leak a little
Probe endpoint used as an unauthenticated DoS vector	Keep it cheap; do not let it trigger expensive work	Reachable from anywhere in the cluster until Day 4
Probe path reachable externally	Never expose <code>/healthz</code> through the Ingress	—

12. Performance considerations

- With 34 pods × 3 probes, the kubelet generates roughly **1,000 probe requests per minute** cluster-wide. Keep each one cheap.
- A readiness probe that performs a real database query multiplies that load onto the database.
- `SILENT_PATHS` in `axispay_common/metrics.py` excludes probe paths from access logging. Without it, probe traffic would be the single largest contributor to Loki storage on Day 5, burying real traffic.
- `timeoutSeconds` must exceed the worst-case response, or you get spurious failures under load — precisely when you least want them.

13. High availability

Probes are what make replica count meaningful. Three replicas where two cannot serve is one replica's worth of availability — and without readiness, Kubernetes cannot tell the difference. Readiness is also what makes rolling updates, node drains and PodDisruptionBudgets work correctly.

14. Disaster recovery

During a dependency outage, correct readiness probes mean pods leave rotation and rejoin automatically when the dependency returns — **no human action at all**. Incorrect liveness probes mean a thundering herd of restarts that extends the outage well beyond the original fault.

15. Monitoring

Metric	Threshold
<code>kube_pod_status_ready{condition="false"}</code>	Sustained > 2 min
<code>kube_pod_container_status_restarts_total</code> rate	Any sustained increase
<code>prober_probe_total{result="failed"}</code>	Rising
Ready replicas vs desired	Gap > 5 min — the single best deployment alert

16. Troubleshooting

Symptom	Cause	Command	Fix
All replicas restart together	Liveness checks a dependency	<code>kubectl get deploy -o yaml</code> <code>\ grep -A3 livenessProbe</code>	Point liveness at <code>/healthz</code>
Pod never Ready	Readiness dependency unavailable	<code>kubectl exec ... -- wget -qO- localhost:8080/readyz</code>	Fix the dependency
CrashLoop on startup	Liveness racing a slow start	<code>kubectl describe pod</code> → probe failed	Add/extend <code>startupProbe</code>
connection refused on probe	Wrong port, or not listening yet	<code>kubectl exec ... -- ss -ltn</code>	Fix the port; add startup probe
Probes pass, traffic fails	Probe healthier than reality	Test the business endpoint	Make <code>/readyz</code> meaningful
Rollout hangs	New pods never Ready	<code>kubectl describe pod</code> <code><new></code>	Read the probe failure — the rollout is correctly refusing

Interview questions

- What is the difference between liveness and readiness?** *Consequence. Liveness failure restarts the container; readiness failure removes it from Service endpoints without restarting. Liveness must never check dependencies; readiness should.*
- Why must liveness never check a database?** *Because a brief database outage would fail liveness on every replica simultaneously, restarting the entire service. Restarting does not fix the database — it discards warm processes and connection pools and turns a dependency blip into a total outage.*
- What does a startup probe do that the others cannot?** *It suspends liveness entirely until it passes, giving a slow-starting container a generous budget without weakening liveness for the rest of its life.*
- How long after a pod becomes unready does traffic actually stop?** *(senior)*
`periodSeconds × failureThreshold` (10 s here) plus endpoint propagation to every node's kube-proxy — call it 10–15 s. That gap is why `preStop` exists: the pod must stay up long enough for the removal to reach every node.
- Design readiness for a service with one critical and one optional dependency.** *(senior)* Fail readiness only on the critical one; report the optional one as degraded but stay ready. In AxisPay, `merchant-service` is critical to `payment-service` (no pricing, no payment) but non-critical to `edge-gateway` (only `/account` degrades). That classification is a design decision written in code, not a default.

2.4 Horizontal Pod Autoscaling

1. What it is

A controller that adjusts a Deployment's replica count based on observed metrics.

2. Why it exists

Traffic is not constant. Manual scaling means someone is awake at 06:00 on Black Friday — and, historically, nobody scales back down.

3. The business problem

AxisPay's merchants expect six times normal volume on Black Friday. Last year the team scaled up manually and forgot to scale down for nine days.

4. How it works

```
desiredReplicas = ceil( currentReplicas * currentUtilisation ÷ targetUtilisation )

utilisation = actual usage ÷ the REQUEST
```

The request is the denominator. No request means no denominator: the HPA reports `<unknown>` and does nothing, forever, with no error and no event. This is the arithmetic dependency that makes resources a prerequisite for autoscaling.

Worked example: 4 pods at 140% of request, target 70% → `ceil(4 × 140 / 70) = 8 replicas`.

5. Internal architecture

The HPA controller reconciles every 15 seconds:

1. Fetch metrics from the metrics API (served by metrics-server, which scrapes each kubelet's cAdvisor).
2. Compute utilisation per pod as a percentage of its request.
3. Average across ready pods; apply the formula.
4. Apply `behavior` policies and stabilisation windows.
5. Write `spec.replicas` on the Deployment.

The HPA owns `spec.replicas`. Leaving `replicas` in a manifest that also has an HPA causes them to fight on every `apply`.

6. Component interactions

```
pods -> kubelet/cAdvisor -> metrics-server -> HPA -> Deployment -> ReplicaSet -> pods
```

A closed loop. If metrics-server is down, the whole loop stops silently.

7. Enterprise example

A payments platform scales its authorisation service on CPU and its notification worker on **queue depth** via a custom metric. The distinction matters: authorisation is CPU-bound, but a notification worker is bound by backlog, and CPU tells you nothing about it.

8. Real-world analogy

A supermarket opening more tills as queues lengthen. Tills open quickly when queues form, and close slowly afterwards — because opening and closing repeatedly is worse than leaving one open a few minutes longer.

Where it breaks: a supermarket manager can see the queue. The HPA sees only CPU. If tills are slow because the card terminal is broken, opening more tills does nothing.

9. Best practices

Practice	Reason
Never deploy an HPA without resource requests	No denominator, no autoscaling, no error
Scale up fast, scale down slowly	Prevents flapping
<code>minReplicas</code> ≥ 3 for critical services	Survives a node loss even at minimum
Size the ResourceQuota for <code>maxReplicas</code>	Or scaling stalls during the spike
Remove <code>replicas</code> from HPA-managed manifests	Otherwise HPA and <code>apply</code> fight
Scale on the metric that actually saturates	CPU is often the wrong one
Alert when an HPA sits at <code>maxReplicas</code>	You have run out of headroom

10. Common mistakes

Mistake	Symptom
No CPU request	<code>TARGETS: <unknown></code> , HPA does nothing, no error
Quota sized for steady state	Scaling stops mid-spike with <code>FailedCreate</code>
Scale-down too aggressive	Flapping: scale down → latency rises → scale up
<code>replicas</code> left in the manifest	Replica count oscillates on every apply
Expecting HPA to fix a broken dependency	CPU <i>falls</i> when pods cannot serve; HPA does nothing
Scaling a stateful workload on CPU	New replicas do not share the state — see the velocity bug below

11. Security considerations

Threat	Control	Residual risk
Traffic flood drives cost-amplification scaling	<code>maxReplicas</code> , quota, rate limiting at the edge	An attacker can still drive you to max
HPA scaling a compromised workload wider	Standard workload controls	HPA has no notion of correctness
metrics-server as an attack surface	RBAC; it is cluster-privileged	Compromise yields cluster-wide metrics

12. Performance considerations

- **15-second reconcile interval** plus metrics-server's scrape interval means the HPA reacts on the order of 30–60 seconds. It cannot absorb a sub-minute spike; that is what headroom in `minReplicas` is for.
- **Scale-down stabilisation (300 s)** deliberately trades cost for stability.
- **Pod startup time** is the real limit on how fast you can scale. If a pod takes 45 seconds to become ready, the HPA cannot help you inside that window.

The in-memory state bug — worth understanding properly

`fraud-service` keeps velocity counters in memory. With 6 replicas behind a Service, each pod sees roughly one sixth of the traffic. A rule of "more than 8 attempts in 5 minutes" now effectively fires at **48** attempts.

The fraud control silently weakens as you scale. Nothing errors. No alert fires. Scaling up for performance quietly degrades a security control.

This is why per-replica state and horizontal scaling are fundamentally in tension, and it is exactly what Redis fixes on Day 3.

13. High availability

`minReplicas` is the real HA lever — set it so that losing a node still leaves enough capacity. AxisPay uses `minReplicas: 3` for `payment-service` (one per node) so a node failure never drops below two.

14. Disaster recovery

HPAs are configuration and recover with the manifest. During an incident they can work against you: a service failing slowly may consume *less* CPU, causing the HPA to scale *down* just as demand rises. Knowing when to suspend autoscaling is part of incident response.

15. Monitoring

Metric	Threshold
<code>kube_horizontalpodautoscaler_status_current_replicas == _spec_max_replicas</code>	Sustained — out of headroom
<code>kube_horizontalpodautoscaler_status_condition{condition="ScalingActive",status="false"}</code>	Any — usually <unknown> metrics
Scale events per hour	High = flapping
metrics-server availability	Any gap disables all autoscaling

16. Troubleshooting

Symptom	Cause	Command	Fix
<code>TARGETS: <unknown></code>	No CPU request, or metrics-server down	<code>kubectl describe hpa ;</code> <code>kubectl top pods</code>	Add requests; check metrics-server
Never scales up	Utilisation below target	<code>kubectl describe hpa</code>	More load, or lower the target
Scales up then straight down	Stabilisation too short	<code>kubectl get hpa -o yaml</code>	Lengthen <code>scaleDown</code> window
Stops below <code>maxReplicas</code>	Quota reached	<code>kubectl describe rs ;</code> <code>describe quota</code>	Size the quota for max
Fights your <code>apply</code>	<code>replicas</code> still in the manifest	—	Remove it
<code>FailedGetResourceMetric</code>	metrics-server unhealthy	<code>kubectl -n kube-system</code> <code>logs -l k8s-</code> <code>app=metrics-server</code>	Restart; wait for a scrape window

Interview questions

- Why does an HPA need resource requests?** *Utilisation is a percentage of the request. With no request there is no denominator, so the HPA reports <unknown> and does nothing — silently.*
- Give the formula and work an example.** `desired = ceil(current * utilisation / target)` . Four pods at 140% against a 70% target gives `ceil(8)` = 8 replicas.

3. **Why is scale-down slower than scale-up?** *To prevent flapping. Removing capacity raises latency, which triggers scale-up, which triggers scale-down again. Each cycle costs a pod start and a cold cache. Five minutes of extra capacity is far cheaper than the oscillation.*
 4. **A dependency fails and payments start erroring. What does the HPA do?** *(senior) Nothing, or it scales down — because CPU usage falls when pods cannot serve. Autoscaling responds to load, not correctness. This is the clearest illustration of its boundary.*
 5. **Your service keeps per-user rate-limit counters in memory and is behind an HPA. What breaks?** *(senior) Each replica sees a fraction of the traffic, so the effective limit is multiplied by the replica count. The control silently weakens as you scale, with no error. The fix is shared state — Redis — or consistent hashing so a given key always reaches the same replica.*
-

2.5 DaemonSet, Job and CronJob

1. What it is

Three controllers for workloads a Deployment cannot express: one per node, run-to-completion, and run-on-a-schedule.

2. Why it exists

Not everything is N interchangeable replicas running forever.

3. The business problem

AxisPay needs PCI file-integrity monitoring on **every** node including new ones, a reconciliation run that compares the ledger against the acquirer position and then stops, and settlement at 23:00 **exactly once**.

4. How it works

Controller	Count decided by	Complete when
Deployment	You (<code>replicas</code>)	Never — runs forever
DaemonSet	The node inventory — no <code>replicas</code> field	Never
Job	You (<code>completions</code> , <code>parallelism</code>)	Pod exits 0
CronJob	The schedule	Each Job completes

5. Internal architecture

DaemonSet. The controller watches Nodes, not a replica count. Add a node and a pod appears; drain a node and it goes. Because agents usually must run everywhere — including tainted control-plane nodes — a DaemonSet commonly carries tolerations that ordinary workloads do not.

Job. `backoffLimit` bounds pod retries with exponential back-off. `activeDeadlineSeconds` is a hard wall-clock ceiling. `restartPolicy` must be `Never` or `OnFailure` — `Always` is rejected, because a container that restarts on success could never complete.

CronJob. Creates Jobs on a schedule. Key fields:

Field	AxisPay value	Reason
<code>timeZone</code>	<code>Africa/Johannesburg</code>	The cluster runs UTC; the merchant's end-of-day does not
<code>concurrencyPolicy</code>	<code>Forbid</code>	Settlement must never double-run
<code>startingDeadlineSeconds</code>	600	Run late, but not absurdly late
<code>successfulJobsHistoryLimit</code>	3	Enough to debug; not enough to fill etcd

`timeZone` is not cosmetic. `0 23 * * *` with no `timeZone` on a UTC cluster runs at **01:00 the next day** in Johannesburg. The Tuesday batch runs on Wednesday and may include Wednesday's early transactions. That is an accounting defect, and it is completely silent.

6. Component interactions

```

DaemonSet controller -> watches Nodes -> creates one pod per eligible node
Job controller       -> creates pods -> counts successes -> marks complete/failed
CronJob controller  -> evaluates schedule -> creates a Job -> applies concurrencyPolicy

```

7. Enterprise example

A bank runs four DaemonSets on every node — log shipper, metrics agent, file-integrity monitor, vulnerability scanner — all tolerating every taint, because an unmonitored node is a compliance finding. Settlement runs as a CronJob with `Forbid`, and the application is *also* idempotent, because a Job pod can be killed after doing its work but before recording it.

8. Real-world analogy

DaemonSet: a fire extinguisher on every floor. You do not order "three extinguishers" — you order one per floor, and a new floor gets one automatically. **Job:** the annual stocktake. It starts, finishes, and produces a report. **CronJob:** the cleaner who comes at 23:00 every night.

Where it breaks: if the cleaner is still working at 23:00 tomorrow, `Forbid` means tomorrow's visit is skipped entirely — not queued. For settlement that is correct; for some workloads it is not.

9. Best practices

Practice	Reason
DaemonSet resources tightly capped	An agent that starves its host is worse than no agent
Always set <code>backoffLimit</code> and <code>activeDeadlineSeconds</code>	Bound both retries and wall-clock time
<code>ttlSecondsAfterFinished</code> on Jobs	Otherwise completed Jobs accumulate in etcd
Always set <code>timeZone</code> on a CronJob	The cluster runs UTC
<code>concurrencyPolicy: Forbid</code> for money movement	Two settlement runs would double-count
Make the Job idempotent anyway	Kubernetes gives at-most-once <i>scheduling</i> , not exactly-once <i>execution</i>
<code>updateStrategy.maxUnavailable: 1</code> on DaemonSets	Update one node at a time

10. Common mistakes

Mistake	Symptom
<code>restartPolicy: Always</code> on a Job	Rejected at admission
No <code>activeDeadlineSeconds</code>	A hung nightly job runs until someone notices in the morning
No <code>timeZone</code>	Batch runs on the wrong calendar day, silently
<code>concurrencyPolicy: Allow</code> for settlement	Double-counted money
DaemonSet with no tolerations	Missing on tainted nodes — the ones you most need to watch
Treating Jobs as exactly-once	A pod killed after work but before recording repeats the work
No history limits	Thousands of completed Jobs in etcd

11. Security considerations

Threat	Control	Residual risk
DaemonSets often need host access	Minimise <code>hostPath</code> ; drop capabilities; read-only mounts	Node-level agents are inherently privileged
A compromised DaemonSet runs on every node	Strict RBAC and image provenance	Blast radius is the whole cluster by design
Jobs handling sensitive data leave logs	Short <code>ttlSecondsAfterFinished</code> ; do not log payloads	Logs may already be shipped
CronJob schedule tampering	RBAC on CronJob objects	A schedule change is easy to miss in review

12. Performance considerations

- A DaemonSet multiplies its resource footprint by the node count. 50m × 200 nodes is 10 CPUs of agent.
- `parallelism` and `completions` on a Job control throughput; `completionMode: Indexed` lets each pod take a deterministic slice.
- CronJobs firing at the same minute across many namespaces cause a thundering herd — stagger them.

13. High availability

DaemonSets are inherently distributed — one per node. Jobs are not HA: if the node running a Job pod fails, the pod is recreated elsewhere and the work restarts, which is exactly why idempotency matters. CronJob scheduling depends on the controller-manager; a missed window is governed by `startingDeadlineSeconds`.

14. Disaster recovery

For batch workloads, the recovery question is "**did it run, and did it complete?**" — not "is it up?". Keep a durable record of completion outside Kubernetes (a row in the database), because Job history is pruned. AxisPay's settlement writes its own record for exactly this reason.

15. Monitoring

Metric	Threshold
<code>kube_daemonset_status_number_unavailable</code>	> 0
<code>kube_daemonset_status_desired_number_scheduled</code> vs node count	Mismatch = a taint you do not tolerate
<code>kube_job_status_failed</code>	Any
<code>kube_cronjob_status_last_schedule_time</code>	Older than the schedule interval — it did not fire
Job duration vs historical	Sudden increase

16. Troubleshooting

Symptom	Cause	Command	Fix
DaemonSet missing on a node	Taint not tolerated	<code>kubectl describe node <n></code> → Taints	Add a toleration
Job retries forever	No <code>backoffLimit</code>	<code>kubectl describe job</code>	Set it
Job Error, pod exits non-zero	Application failure	<code>kubectl logs job/<name></code>	Fix the cause
<code>restartPolicy:</code> Always rejected	Jobs forbid it	—	Never or OnFailure
CronJob never fires	Bad schedule or <code>suspend: true</code>	<code>kubectl get cronjob -o yaml</code>	Fix it
CronJob fires at the wrong hour	No <code>timeZone</code>	Same	Set <code>timeZone</code>
Jobs accumulate	No TTL or history limits	<code>kubectl get jobs</code>	Set <code>ttlSecondsAfterFinished</code>

Interview questions

- Why does a DaemonSet have no `replicas` field?** Because the count is determined by the node inventory, not by you. Add a node and a pod appears automatically. The equivalent of "scaling" is changing which nodes are eligible — via `nodeSelector` or tolerations.
- What makes a Job complete?** The pod exiting 0. That is why `restartPolicy: Always` is rejected — a container restarting on success could never finish.
- Why must a CronJob set `timeZone`?** The cluster runs UTC. `0 23 * * *` for a Johannesburg merchant fires at 01:00 the following local day, so the batch settles the wrong calendar day. It is silent and it is an accounting defect.

4. **How would you guarantee a settlement runs exactly once?** *(senior)* You cannot, from Kubernetes alone. `concurrencyPolicy: Forbid` gives at-most-once scheduling, but a Job pod can be killed after doing its work and before recording it, so the retry repeats the work. The guarantee must come from the application: an idempotency key or a uniquely-constrained completion record in the database, checked before starting.
 5. **When would `concurrencyPolicy: Replace` be right, and when catastrophic?** *(senior)* Right for an idempotent refresh where only the latest result matters — a cache warm, a report regeneration. Catastrophic for money movement: it kills a run mid-way, leaving partial work with no record of how far it got.
-

2.6 Rolling updates and graceful shutdown

1. What it is

Replacing every pod of a Deployment with a new version, one controlled step at a time, and terminating the old ones without severing work in flight.

2. Why it exists

Stopping everything and starting the new version is an outage. For a platform that earns per transaction, an outage is revenue that does not arrive.

3. The business problem

Change request CR-2026-0812: `payment-service` v1.0.0 → v1.1.0, during trading, adding fraud scoring and acquirer routing. **No merchant may see a single failed authorisation.**

4. How it works

The Deployment creates a **new ReplicaSet** and shifts replicas across:

Setting	Value	Meaning
<code>maxUnavailable</code>	0	Never fewer than 3 ready pods — capacity never drops
<code>maxSurge</code>	1	At most 4 pods exist at once

With `maxUnavailable: 0` the rollout is strictly **add-then-remove**. It needs headroom for one extra pod and it is slower. On a payment path that is the correct trade.

Readiness is the gate. A new pod counts as available only when its readiness probe passes. Remove the probe and "ready" degrades to "the process started" — and the controller removes an old pod before the new one can serve.

5. Internal architecture

```

t0  RS-A: 3 ready (v1.0.0)                total 3, serving 3
t1  RS-B: +1 pod created, not ready        total 4, serving 3
t2  new pod passes READINESS, joins endpoints total 4, serving 4
t3  one old pod terminates (preStop, drain) total 3, serving 3
...  repeat
t5  RS-B: 3 ready; RS-A: 0 replicas, RETAINED total 3, serving 3

```

`progressDeadlineSeconds: 300` marks the Deployment failed if it has not progressed — without it a broken release sits "in progress" forever and no pipeline ever fails.

6. Component interactions — termination

Two things happen **in parallel** when a pod is deleted:

#	Event	Why it matters
1	Endpoint controller removes the address; kube-proxy reprograms on every node	Eventually consistent — it takes time to propagate
2	<code>preStop</code> runs (<code>sleep 8</code>)	The pause that lets step 1 reach every node
3	SIGTERM to PID 1	App marks itself unready and finishes in-flight work
4	Up to <code>terminationGracePeriodSeconds</code> (45 s)	Must exceed the longest in-flight operation
5	SIGKILL	Cannot be caught. Anything still running is severed.

The ENTRYPOINT form matters. With the shell form, PID 1 is `/bin/sh`, which does **not** forward SIGTERM. Kubernetes waits the full grace period and then SIGKILLS every pod on every rollout — turning zero-downtime into guaranteed-downtime, with no error anywhere. Every AxisPay Dockerfile uses the exec form for this reason.

7. Enterprise example

A payments platform deploys 40 times a day with `maxUnavailable: 0`, a `PodDisruptionBudget` on every workload, and anti-affinity across zones. A failed readiness probe halts the rollout automatically and the previous version keeps serving. The release fails safe, without a human deciding anything.

8. Real-world analogy

Replacing staff mid-shift without leaving the counter unstaffed: bring the new person in, wait until they are actually ready to serve, then let one of the old team finish their current customer and leave. Repeat.

Where it breaks: a human can say "give me two minutes to finish". A container gets SIGTERM and a fixed grace period, then SIGKILL regardless.

9. Best practices

Practice	Reason
<code>maxUnavailable: 0</code> for user-facing services	Capacity never drops
Always define a readiness probe	Without it the rollout is not zero-downtime, whatever the strategy says
Set <code>progressDeadlineSeconds</code>	A stuck rollout must eventually fail
<code>preStop</code> pause before SIGTERM	Lets endpoint removal propagate
Grace period > longest in-flight operation	Not the average
Exec-form <code>ENTRYPOINT</code>	Or SIGTERM never reaches your process
Record <code>kubernetes.io/change-cause</code>	<code>rollout history</code> becomes readable in an incident
Measure the rollout	A release you did not measure proves nothing

10. Common mistakes

Mistake	Symptom
No readiness probe	Traffic dropped during every rollout
<code>maxUnavailable > 0</code> on a critical path	Capacity dips under load
Shell-form <code>ENTRYPOINT</code>	Graceful shutdown silently never happens
Grace period too short	In-flight payments severed by SIGKILL
No <code>preStop</code>	502s at the moment of termination
Changing only <code>spec.replicas</code> and expecting a rollout	Only <code>spec.template</code> changes create a ReplicaSet
Relying on <code>rollout undo</code> beyond <code>revisionHistoryLimit</code>	Nothing to roll back to

11. Security considerations

Threat	Control	Residual risk
Rollback re-introduces a vulnerable image	Scan on pull; block bad digests at admission	Emergency rollback may bypass policy
Anyone with <code>update</code> on Deployments runs arbitrary images	RBAC; registry allow-listing at admission	CI credentials remain powerful
In-flight data lost on abrupt termination	Grace period + <code>preStop</code> + idempotency	SIGKILL is always possible

12. Performance considerations

- Rollout duration $\approx \text{replicas} \times (\text{pod startup} + \text{readiness delay}) \div \text{maxSurge}$. A high `initialDelaySeconds` makes every release slow.
- `maxUnavailable: 0` requires the cluster to fit `replicas + maxSurge` — real capacity, not theoretical.
- With 200 replicas, use a **percentage** rather than an absolute surge.
- Old ReplicaSets consume etcd and API server memory. Keep `revisionHistoryLimit` modest.

13. High availability

Rolling updates are only zero-downtime when readiness probes, `maxUnavailable: 0`, sufficient capacity **and** graceful shutdown are all in place. Missing any one of them silently degrades the guarantee — and the failure appears as sporadic 502s during deploys, which is easy to dismiss.

14. Disaster recovery

`kubectl rollout undo` is near-instant because the old ReplicaSet is retained at zero replicas — no image pull, no new object. Beyond `revisionHistoryLimit`, recovery means re-applying an older manifest from Git.

Rollback is only safe if the change is backwards-compatible. A release that migrated a database schema cannot be rolled back by scaling a ReplicaSet — which is why the Day 5 capstone separates the schema migration from the application rollout.

15. Monitoring

Metric	Threshold
<code>kube_deployment_status_replicas_available</code> VS <code>_spec_replicas</code>	Gap > 5 min
<code>kube_deployment_status_observed_generation</code> VS <code>metadata.generation</code>	Lagging = controller not reconciling
Error rate during a deploy window	Any increase — this is what loadgen measures
Two ReplicaSets non-zero for a long period	Stuck rollout
Pod termination duration	Approaching the grace period

16. Troubleshooting

Symptom	Cause	Command	Fix
Rollout stuck part-way	New pods never Ready	<code>kubectl describe pod <new></code>	Read the probe failure
ProgressDeadlineExceeded	Exceeded the deadline	<code>kubectl describe deploy</code>	Investigate, then <code>rollout undo</code>
Errors during rollout	No readiness probe or <code>maxUnavailable > 0</code>	<code>kubectl get deploy -o yaml</code>	Add readiness; set 0
Rollout does not start	Nothing in <code>spec.template</code> changed	<code>kubectl rollout history</code>	Change the template
Pods take the full grace period to die	SIGTERM not reaching the process	Check <code>ENTRYPOINT</code> form	Use exec form
502s at termination	Missing/short <code>preStop</code>	Check <code>lifecycle.preStop</code>	Add or extend the pause
Cannot roll back	History pruned	<code>kubectl rollout history</code>	Re-apply from Git

Interview questions

- What makes a rolling update zero-downtime?** A readiness probe plus `maxUnavailable: 0`. The probe means "ready" reflects real serving capability; `maxUnavailable: 0` means no old pod is removed until a new one is genuinely ready.
- replicas: 3, maxSurge: 1, maxUnavailable: 0. Maximum pods, minimum serving?** Maximum 4, minimum serving 3. With `maxSurge: 0` as well, the rollout could never start — Kubernetes rejects that combination.
- Describe the termination sequence.** Pod marked Terminating and endpoint removal begins in parallel; `preStop` runs; SIGTERM to PID 1; up to `terminationGracePeriodSeconds` to exit; then SIGKILL. `preStop` exists because endpoint removal is eventually consistent across nodes.
- Why does the ENTRYPOINT form affect graceful shutdown?** (senior) Shell form makes `/bin/sh` PID 1, and it does not forward SIGTERM to the application. The process never learns it should drain, Kubernetes waits the full grace period, then SIGKILLs. Every rollout silently severs in-flight work, with no error anywhere.
- When is Recreate the correct strategy?** (senior) When two versions cannot run simultaneously — a schema migration that is not backwards-compatible, or a singleton holding an exclusive lock. It is a deliberate outage in exchange for correctness, and it should be a conscious decision rather than a default.

2.7 Java Memory Management and JVM Tuning in Kubernetes

1. What it is

The discipline of sizing a Java process **inside a container** rather than on a bare VM. In Kubernetes, the JVM does not own the whole machine. It lives under a cgroup memory ceiling, and the Java heap is only one consumer under that ceiling.

2. Why it exists

The JVM can be perfectly healthy and still be **OOMKilled by the container runtime**. Teams that think "`-Xmx` = memory usage" learn very quickly that metaspace, thread stacks, JIT code cache, direct buffers and JNI allocations do not live inside the Java heap.

3. The business problem

AxisPay's `fraud-service` passed every functional test and every canary check. Yet every few hours, under genuine merchant load, one replica disappeared with no Java exception and no useful application log. Then the second replica died. Then the HPA added cold pods that took nearly a minute to warm up, and the fraud decision path slowed enough to push authorisation latency beyond the SLO.

The deployment looked reasonable:

Setting	Value
Container memory limit	2Gi
JVM flag	<code>-Xmx2g</code>
Replicas	3
JDK	17

Someone had set the heap equal to the container limit because they "did not want to waste memory". In reality they had left **zero headroom** for everything that is not heap.

4. How it works

Kubernetes enforces memory with the container's cgroup limit. The JVM, since JDK 10, is container-aware and can size itself relative to that limit — **if you let it**.

```

container memory limit = 3Gi

inside that 3Gi:
  Java heap           ~70%
  metaspace           variable
  thread stacks       threads × -Xss
  direct buffers      NIO / Netty / TLS
  code cache + GC     variable
  libc / JVM native    variable

cross 3Gi total -> kernel OOM kill -> exit 137

```

Two approaches dominate:

Approach	How it works	Risk
Hard-coded <code>-Xmx</code>	Fixed heap size regardless of the container	Easy to set too high and forget native headroom
<code>-XX:MaxRAMPercentage</code>	Heap is a percentage of container memory	Safer in containers; scales with the limit

For containerised Java, `-XX:MaxRAMPercentage` is usually the better default because it keeps the JVM's idea of usable heap tied to the cgroup it is actually running under.

5. Internal architecture

The failure boundary is not the heap. It is the **cgroup**.

Layer	Responsibility	What it sees
Kubernetes manifest	Declares <code>requests.memory</code> and <code>limits.memory</code>	YAML
kubelet / container runtime	Writes cgroup memory limits	Total process memory
JVM ergonomics	Decides heap, GC threads, compiler threads	Container-aware view of RAM and CPU
Linux kernel	Kills the process when total memory exceeds limit	RSS / working set, not "heap used"

Important memory areas for a Spring Boot service:

Memory area	Inside heap?	Why it grows
Object heap	Yes	Request objects, caches, ORM entities
Metaspace	No	Loaded classes, frameworks, proxies
Thread stacks	No	Each request pool / scheduler / GC / JIT thread
Direct buffers	No	NIO, TLS, HTTP clients, Netty, gRPC
Code cache	No	JIT-compiled machine code

This is why "heap used below 80%" and "pod OOMKilled" can both be true at the same time.

6. Component interactions

```
Deployment -> container limit 3Gi
JVM        -> MaxRAMPercentage=70 => heap target ~2.1Gi
Spring     -> starts threads, loads classes, opens pools
traffic    -> allocates heap + direct buffers
kernel     -> observes total process memory
if total > 3Gi -> OOMKill (137)
```

The JVM sees the container limit, but the kernel kills on **total** memory, not heap usage. That distinction explains most "mystery" Java OOMs in Kubernetes.

7. Enterprise example

A payments platform standardises Java container settings in a shared Helm chart:

Environment	Memory limit	JVM policy
Small internal service	768Mi	-XX:MaxRAMPercentage=60
Payment path	2Gi to 4Gi	-XX:MaxRAMPercentage=65-70
Batch reconciler	4Gi+	-Xms closer to -Xmx , throughput bias acceptable

Teams can override, but the default is opinionated: **leave headroom first, then tune the heap**. That policy eliminated most OOMKilled incidents not by making services smaller, but by making the failure mode explicit.

8. Real-world analogy

A container memory limit is the total luggage allowance for a flight. The Java heap is the suitcase you planned for clothes. Metaspace, toiletries, a laptop and duty-free purchases are all the other bags you still carry. Filling the suitcase to the exact airline limit does not mean you are safe; it means the first extra kilogram at check-in gets rejected.

Where it breaks: an airline weighs at the counter, once. The kernel continuously enforces the limit, and the JVM's native memory can grow after the process has been running for hours.

9. Best practices

Practice	Reason
Size the container first, then the heap	The heap is only one consumer under the cgroup limit
Prefer <code>-XX:MaxRAMPercentage</code> over a blindly copied <code>-Xmx</code>	Safer and self-adjusting across environments
Leave 25-35% headroom outside the heap for Spring Boot services	Metaspace, threads, direct buffers and JIT all need space
Set <code>-Xms</code> lower than <code>-Xmx</code> for bursty services	Faster scheduling, lower idle footprint
Cap thread counts deliberately	Hundreds of threads mean hundreds of stacks outside the heap
Measure off-heap consumers under load	Heap graphs alone do not explain container OOMs
Keep a consistent GC across replicas	Mixed GC behaviour makes latency hard to reason about
Document the JVM flags in the manifest, not only in Dockerfile lore	Incidents start with <code>kubectl get deploy -o yaml</code>

10. Common mistakes

Mistake	Symptom
<code>-Xmx</code> equal to the memory limit	<code>OOMKilled</code> , exit 137, heap graphs look "fine"
Copying VM-era JVM flags into containers	Wrong heap size and wrong CPU assumptions
Ignoring direct memory	TLS / HTTP clients fail only under load
Setting <code>-Xms = -Xmx</code> on every service	High idle memory, harder bin-packing
Chasing Java heap leaks when the leak is native	Heap dump looks normal; pod still dies
Tiny memory limit with many request threads	Low throughput and random OOMs during traffic spikes

11. Security considerations

Threat	Control	Residual risk
Heap dumps expose PAN-adjacent or token data	Restrict dump generation and storage; scrub sensitive fields	A crash dump is still sensitive data at rest
Exposed JMX / debug ports leak internals	Disable or bind to localhost; protect with NetworkPolicy	Misconfigured sidecars can still expose them
OOM as an availability attack	Enforce limits and right-size headroom	A single expensive request pattern can still drive memory up
Unsafe diagnostic flags in prod	Standardise approved JVM options	Break-glass changes still happen during incidents

12. Performance considerations

GC choice is a latency trade, not a religion.

GC	Strength	Weakness	AxisPay fit
G1GC	Predictable pauses, balanced default on JDK 17	Some throughput cost versus Parallel	Best default for payment APIs
Parallel GC	High throughput	Longer stop-the-world pauses	Good for batch Jobs, risky for p99 latency
ZGC	Very low pause times even on large heaps	More operational complexity; not always needed for modest heaps	Consider for very latency-sensitive or large-memory services

- `payment-service` and `fraud-service` care about p99 latency more than raw throughput, so G1GC is usually the right starting point.
- Parallel GC can process more work per CPU in batch-style workloads, but its pause profile is a poor fit for synchronous payment authorisation.
- ZGC is attractive when heap sizes are large enough that even G1 pauses become a business problem, but it should be chosen because measured latency demands it, not because it is fashionable.

13. High availability

Right memory sizing is an HA control. A pod that dies every few hours under peak load is not "mostly available" — it is systematically removing redundancy exactly when the system needs it most. JVM tuning also affects rollout safety: if the new version uses more native memory than the old one, a rollout can fail only at scale, after half the pods are already replaced.

14. Disaster recovery

JVM flags recover from Git with the manifest, but evidence of the failure may not. OOMKilled pods can disappear before anyone inspects them. During an incident, capture:

1. `kubectl describe pod`
2. `kubectl top pod --containers`
3. JVM metrics around heap, GC and threads
4. The exact runtime flags from `kubectl get pod -o yaml`

If you do not preserve those four, the next restart erases the best clues.

15. Monitoring

Metric / signal	Threshold	Why it matters
<code>container_memory_working_set_bytes / limit</code>	> 85% sustained	Container-level risk regardless of heap view
<code>jvm_memory_used_bytes{area="heap"} / jvm_memory_max_bytes</code>	> 75-80% sustained	Heap pressure trend
<code>jvm_gc_pause_seconds p99</code>	Rising	Memory pressure often appears as pause inflation first
<code>jvm_threads_live_threads</code>	Sudden growth	Thread stacks consume native memory
<code>kube_pod_container_status_last_terminated_reason{reason="OOMKilled"}</code>	Any	Page immediately

16. Troubleshooting

Worked incident: `fraud-service` OOMKilled every few hours.

```
kubectl top pod -n axispay-core fraud-service-7c8c7d9d4d-x2v9p
```

NAME	CPU (cores)	MEMORY (bytes)
fraud-service-7c8c7d9d4d-x2v9p	420m	1886Mi


```
kubectl describe pod -n axispay-core fraud-service-7c8c7d9d4d-x2v9p
```

```
Last State:      Terminated
Reason:          OOMKilled
Exit Code:       137
```

The manifest had:

```
resources:
  requests:
    memory: "1536Mi"
  limits:
    memory: "2Gi"
env:
- name: JAVA_TOOL_OPTIONS
  value: "-Xmx2g -XX:+UseG1GC"
```

The fix:

```
resources:
  requests:
    memory: "2Gi"
  limits:
    memory: "3Gi"
env:
- name: JAVA_TOOL_OPTIONS
  value: "-XX:MaxRAMPercentage=70 -XX:+UseG1GC"
```

Why it worked:

Before	After
Heap fixed at 2g inside 2Gi limit	Heap allowed to grow to ~70% of 3Gi
No room for metaspace / threads / direct buffers	~900Mi headroom outside heap
OOMKilled, exit 137	Stable under the same load profile

Additional checks:

Symptom	Cause	Command	Fix
Heap low, container memory high	Off-heap growth	Compare <code>container_memory_*</code> with <code>jvm_memory_*</code>	Reduce thread count, direct memory, or raise limit
OOM only during startup	Spring context spikes	<code>kubectl describe pod ; startup metrics</code>	Raise startup headroom; review <code>-Xms</code>
GC pauses spike before OOM	Heap too tight	JVM metrics / GC logs	Increase limit or lower allocation rate
Memory flat, restarts continue	Wrong diagnosis; maybe liveness or node pressure	<code>describe pod ; node events</code>	Fix the real cause

Interview questions

- Why can a Java pod be OOMKilled even when the heap is not full?** *Because the container limit applies to total process memory, not only the heap. Metaspace, thread stacks, direct buffers, JIT code cache and other native allocations all count toward the cgroup limit.*
- Why is `-XX:MaxRAMPercentage` usually safer than hard-coded `-Xmx` in Kubernetes?** *Because it sizes the heap relative to the container's memory limit, which is the real boundary in Kubernetes. It reduces the chance that someone copies an old `-Xmx` into a smaller or larger container and leaves the wrong headroom.*
- What is wrong with `-Xmx2g` inside a `2Gi` container?** *It leaves effectively no space for non-heap memory. The JVM may run for a while, but under enough class loading, threads or direct buffer usage the kernel kills it with exit 137.*
- When would you prefer G1GC, Parallel GC or ZGC?** *(senior) G1GC for general low-latency service workloads, Parallel GC for throughput-oriented batch processing where pause time matters less, and ZGC when very low pause times on larger heaps are a measured business need. The right choice comes from latency and throughput goals, not ideology.*
- A service is OOMKilled and `kubectl top` never shows it near the limit. What happened?** *(senior) `kubectl top` is a scrape of recent usage, not a continuous trace. A transient spike in total process memory can exceed the cgroup limit between samples. That is why termination reason, container memory metrics and JVM metrics together matter more than one top snapshot.*

2.8 Metrics Collection from Java Applications

1. What it is

The path from a Spring Boot application's internal view of itself to Prometheus and alerting. In AxisPay's Java services, **Micrometer** is the instrumentation facade, Spring Boot Actuator exposes the metrics, and Prometheus scrapes them.

2. Why it exists

Kubernetes tells you what the container is doing. The JVM tells you what the Java process is doing. You need both views, because "memory high" means something very different depending on whether the pressure is in-heap or off-heap.

3. The business problem

A merchant reports intermittent authorisation slowdowns. `kubect1 top pod` shows `payment-service` memory rising. The on-call engineer sees only a container-level graph and assumes a Java heap leak. The team prepares for a cache rollback.

But Prometheus shows heap usage flat while daemon threads and direct memory climb. The real cause is a misconfigured HTTP client pool to `merchant-service`, not a heap leak at all. Without JVM metrics, the team would have rolled back the wrong change and learned nothing.

4. How it works

Micrometer gives Spring Boot one API for counters, timers, gauges and distribution summaries. The Prometheus registry converts those measurements into the text format Prometheus scrapes at `/actuator/prometheus`.

Typical Spring Boot configuration:

```
management:
  endpoints:
    web:
      exposure:
        include: health,info,prometheus
  endpoint:
    health:
      probes:
        enabled: true
```

Dependencies usually include:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-actuator</artifactId>
</dependency>
<dependency>
  <groupId>io.micrometer</groupId>
  <artifactId>micrometer-registry-prometheus</artifactId>
</dependency>
```

Once exposed, Prometheus reaches the endpoint either through annotations or an operator-managed `ServiceMonitor`.

5. Internal architecture

Layer	Role	Example
Application code	Emits business metrics	<code>payment_authorized_total</code>
Micrometer	Facade and registry integration	Counter, Timer, Gauge
Spring Boot Actuator	HTTP exposure	<code>/actuator/prometheus</code>
Prometheus	Scrapes and stores time series	15-second scrape
Alertmanager / Grafana	Alerts and visualisation	p99 GC pause alert

JVM metrics Micrometer exposes out of the box include:

Metric family	Question it answers
<code>jvm_memory_*</code>	How much heap / non-heap is used?
<code>jvm_gc_pause_seconds_*</code>	How often and how long is GC stopping the world?
<code>jvm_threads_*</code>	Are thread counts growing abnormally?
<code>jvm_classes_*</code>	Is class loading stable or exploding?
<code>process_cpu_usage</code>	How busy is the JVM process itself?

6. Component interactions

```
Spring Boot app
  -> Micrometer collects JVM + app metrics
  -> /actuator/prometheus exposes text
Prometheus
  -> scrapes the Service / pod
Grafana / Alertmanager
  -> queries and alerts
operator / on-call
  -> correlates JVM and container metrics
```

The critical diagnostic pair is:

Metric	Point of view
<code>container_memory_working_set_bytes</code>	cgroup / container view
<code>jvm_memory_used_bytes</code>	JVM view

If container memory is high but heap usage is moderate, the problem is likely **off-heap**.

7. Enterprise example

A large payments platform publishes one standard dashboard per Java service:

Panel	Why it is there
Request rate, error rate, latency	Service-level correctness and SLO
Heap used / max	In-heap pressure
GC pause p50 / p95 / p99	Latency impact of memory pressure
Live threads / daemon threads	Pool leaks and runaway executors
Container memory vs heap used	Off-heap diagnosis
CPU throttling ratio	Whether latency is compute or quota related

The same dashboard works across `payment-service`, `fraud-service`, `auth-service` and `merchant-service` because the instrumentation contract is standardised.

8. Real-world analogy

A building has both an electricity meter for the whole property and circuit-level meters for the server room, kitchen and HVAC. The building meter tells you total consumption; the circuit meters tell you where it is going. Troubleshooting with only the building meter leads to expensive guesses.

Where it breaks: a JVM is not cleanly partitioned like electrical circuits. Some native memory use is harder to attribute precisely, which is why you correlate trends rather than expect a perfect one-to-one accounting.

9. Best practices

Practice	Reason
Expose <code>/actuator/prometheus</code> only inside the cluster	Metrics are operationally useful but not public data
Use a shared Micrometer naming convention	Dashboards and alerts become reusable
Scrape both container and JVM metrics	One view alone is incomplete
Add business metrics alongside JVM metrics	"Healthy JVM" is not the same as "payments succeeding"
Keep label cardinality low	Merchant ID and PAN-like identifiers must never become labels
Prefer <code>ServiceMonitor</code> where the Prometheus Operator exists	Declarative and consistent
Alert on trends, not single blips	GC and heap vary naturally under load

10. Common mistakes

Mistake	Symptom
Exposing Actuator but not Prometheus registry	<code>/actuator/prometheus</code> returns 404
Scraping only container metrics	Heap leak and off-heap leak look identical
High-cardinality labels	Prometheus memory usage explodes
Alerting on raw heap used without <code>heap_max</code>	False alarms across differently sized pods
Forgetting non-heap metrics	Thread / metaspace issues go unseen
Exposing metrics through the public Ingress	Unnecessary information leakage

11. Security considerations

Threat	Control	Residual risk
Sensitive data in metric labels	Strict review; never label by merchant, account or token	A developer can still add a bad label
Metrics endpoint exposed externally	Cluster-only Service, no Ingress, NetworkPolicy	Internal attackers may still reach it
Operational reconnaissance via metrics	Least-privilege access to dashboards and Prometheus	Some metadata remains inherently visible
Prometheus scrape of every pod increases blast radius	Namespace scoping and RBAC	Monitoring remains a privileged component

12. Performance considerations

- Metric collection is not free, but Micrometer's overhead is typically modest when labels are controlled.
- Histograms and percentiles increase cardinality and storage cost; enable them where they answer a real question.
- Scrape intervals matter. Fifteen seconds is common; shorter intervals improve fidelity but increase load.
- For JVM diagnosis, `container_memory_working_set_bytes` is usually more meaningful than RSS because it better approximates actively used memory under cgroups.

Most important JVM alerts for a payments workload:

Metric	Suggested signal	Why it matters
<code>jvm_gc_pause_seconds</code> p99	Rising or above SLO budget	Direct user latency impact
<code>jvm_memory_used_bytes{area="heap"} / jvm_memory_max_bytes</code>	> 0.80 sustained	Heap pressure before OOM
<code>jvm_threads_daemon_threads</code>	Sudden increase	Thread leak / pool mis-sizing
<code>jvm_classes_loaded_classes</code>	Unusual growth	Proxy / classloader issues
<code>container_memory_working_set_bytes / limit</code>	> 0.85 sustained	Container-level kill risk

13. High availability

Metrics do not make a service available, but they make correct fail-safe action possible. Without JVM metrics, teams restart healthy pods and leave broken ones untouched. Observability is what lets autoscaling, right-sizing and rollout safety become engineering rather than guesswork.

14. Disaster recovery

During an incident, dashboards preserve the history that pod restarts erase. When a `fraud-service` pod is recreated, its in-process counters reset. Prometheus does not. That historical continuity is why a post-incident review can answer "did memory climb for hours, or spike for seconds?" instead of telling stories from memory.

15. Monitoring

Example scrape with annotations:

```
metadata:
  annotations:
    prometheus.io/scrape: "true"
    prometheus.io/port: "8080"
    prometheus.io/path: "/actuator/prometheus"
```

Example with a `ServiceMonitor` :

```
apiVersion: monitoring.coreos.com/v1
kind: ServiceMonitor
metadata:
  name: payment-service
spec:
  selector:
    matchLabels:
      app: payment-service
  endpoints:
    - port: http
      path: /actuator/prometheus
      interval: 15s
```

Correlation patterns worth memorising:

Pattern	Interpretation
Container up, heap up with it	Likely in-heap pressure
Container up, heap flat	Off-heap / threads / direct memory / metaspace
Heap flat, GC pause up	Heap near pressure threshold or CPU throttling affecting GC
Heap down after GC, container still high	Native memory remains elevated

16. Troubleshooting

Symptom	Cause	Command / query	Fix
<code>/actuator/prometheus</code> 404	Registry missing or endpoint not exposed	<code>curl localhost:8080/actuator/prometheus</code>	Add Micrometer Prometheus registry; expose endpoint
Prometheus target down	Wrong port/path/Service labels	<code>kubectl describe servicemonitor ;</code> Prometheus targets page	Fix discovery config
High container memory, normal heap	Off-heap usage	Compare <code>container_memory_working_set_bytes</code> with <code>jvm_memory_used_bytes</code>	Inspect threads, direct memory, pools
Alert flood from one metric	Bad label cardinality	Prometheus TSDB / query inspection	Remove offending labels
JVM looks healthy, payments still failing	Missing business metrics	Query error rate and dependency metrics	Instrument the actual payment path

Interview questions

- What role does Micrometer play in a Spring Boot application?** *It is the metrics facade. Application and framework code record counters, timers and gauges against Micrometer, and a registry such as Prometheus determines how those metrics are exported.*
- Why do you need both container metrics and JVM metrics?** *Because they answer different questions. Container metrics show cgroup reality — what the kernel may kill. JVM metrics show what the Java process thinks is happening inside heap, GC, threads and class loading.*
- What does `/actuator/prometheus` expose?** *A text representation of application, framework and JVM metrics in Prometheus scrape format, typically served by Spring Boot Actuator with the Prometheus Micrometer registry on the classpath.*
- How do you diagnose off-heap memory pressure?** *(senior) Compare `container_memory_working_set_bytes` against `jvm_memory_used_bytes`. If container memory rises while heap remains comparatively flat, the pressure is outside the heap — often direct buffers, threads, metaspace or other native allocations.*
- Why are high-cardinality labels dangerous in a payments platform?** *(senior) They explode time-series count, increase Prometheus memory and query cost, and may accidentally embed sensitive business identifiers. Labels must describe dimensions you aggregate by, not individual transactions or merchants.*

2.9 Health Probes for Spring Boot Actuator

1. What it is

Using Spring Boot Actuator's health groups as first-class Kubernetes probe endpoints: `/actuator/health/liveness` for process health and `/actuator/health/readiness` for traffic eligibility.

2. Why it exists

The generic `/actuator/health` endpoint is usually too broad for Kubernetes probes. A pod can be alive but temporarily unable to serve because a dependency is down. Kubernetes needs those consequences separated cleanly.

3. The business problem

AxisPay had a routine database failover. It lasted forty seconds. That should have caused transient payment errors and a quick recovery.

Instead, every `payment-service` pod restarted.

Why? Both liveness and readiness pointed at the same generic `/actuator/health` endpoint. That endpoint included the database check. When the database slowed, readiness correctly wanted the pod out of rotation — but liveness also failed, so the kubelet restarted the process, dropped caches, severed in-flight work and amplified a dependency blip into a platform incident.

4. How it works

Spring Boot 2.3+ has native Kubernetes probe groups. When enabled, Actuator exposes separate endpoints whose semantics match Kubernetes more closely:

```
management:
  endpoint:
    health:
      probes:
        enabled: true
```

Resulting endpoints:

Endpoint	Intended for	Should check dependencies?
<code>/actuator/health/liveness</code>	<code>livenessProbe</code>	No
<code>/actuator/health/readiness</code>	<code>readinessProbe</code>	Yes, critical ones

Kubernetes probe example:

```

livenessProbe:
  httpGet:
    path: /actuator/health/liveness
    port: 8080
  periodSeconds: 10
  failureThreshold: 3

readinessProbe:
  httpGet:
    path: /actuator/health/readiness
    port: 8080
  periodSeconds: 5
  failureThreshold: 2

```

5. Internal architecture

Actuator aggregates health indicators into groups.

Group	Typical members	Meaning
Liveness	Internal state only	"Should the process be restarted?"
Readiness	Internal state + critical dependencies	"Should traffic come here right now?"

A Spring Boot service may include indicators for:

Indicator	Liveness?	Readiness?
Application state	Yes	Yes
Disk space	Often yes	Often yes
PostgreSQL	No	Yes
Redis used for shared counters	No	Yes
Optional analytics dependency	No	Usually no

That separation is the design work. The framework can expose the endpoints, but only the team knows which dependencies are critical to the business operation.

6. Component interactions

```

Spring Boot Actuator
  -> computes liveness / readiness groups
kubelet
  -> calls the endpoints locally
readiness fails
  -> pod Ready=false -> removed from endpoints
liveness fails
  -> container restarted
startupProbe active
  -> liveness suppressed until startup succeeds

```

The probe path is local to the pod. A Service, Ingress or load balancer is not involved.

7. Enterprise example

A bank standardises Spring Boot probe behaviour with one internal starter library:

Standard	Purpose
Actuator enabled	Common operational surface
Probe groups enabled	Correct Kubernetes semantics
Readiness includes DB, message broker and Redis only when critical	Business-driven dependency classification
Liveness excludes all downstream checks	Prevent cascading restarts

This means `auth-service`, `merchant-service`, `core-service` and `payment-service` all behave consistently during incidents, which matters more than each team independently rediscovering the same lessons.

8. Real-world analogy

A cashier can be alive but not ready to take customers because the till has lost its network connection. You stop sending new customers to that till, but you do not send the cashier home and hire a replacement. Liveness is "is the cashier capable of working at all?" Readiness is "can this till serve the next customer right now?"

Where it breaks: a human cashier can explain nuance. A health endpoint reduces that nuance to success or failure, so the dependency classification behind readiness must be thought through carefully.

9. Best practices

Practice	Reason
Enable probe groups with <code>management.endpoint.health.probes.enabled=true</code>	Avoids overloading generic health semantics
Point liveness at <code>/actuator/health/liveness</code>	Prevents dependency blips from causing restarts
Point readiness at <code>/actuator/health/readiness</code>	Lets traffic drain when critical dependencies fail
Tune readiness to react faster than liveness	Removing from rotation is cheaper than restart
Use <code>startupProbe</code> for slow Spring context initialisation	Liveness should not race startup
Include only critical dependencies in readiness	Optional systems should degrade, not eject the pod
Test failover drills against probe behaviour	A good config on paper is not enough

10. Common mistakes

Mistake	Symptom
Liveness and readiness both use <code>/actuator/health</code>	Cascading restarts during DB or Redis blips
Readiness excludes a truly critical dependency	Pod stays ready while payments fail
Liveness includes database checks	Dependency issue becomes platform restart storm
No startup probe on a slow service	CrashLoopBackOff during cold start
Probe groups not enabled	404 on group endpoints
Readiness too slow	Pod keeps taking traffic after it should drain

11. Security considerations

Threat	Control	Residual risk
Health endpoints reveal dependency names and status	Show status with minimal detail; avoid public exposure	Internal callers still learn topology
Probe endpoints exposed through Ingress	Keep Actuator internal-only	Misrouted paths can still leak them
Health checks call privileged subsystems	Keep them read-only and cheap	A badly written indicator can still do too much

12. Performance considerations

- Health indicators run often, so they must be lightweight.
- A readiness check that runs a real SQL query on every pod every 5 seconds can become noticeable load in large fleets.
- Spring context initialisation for JVM services is not instantaneous. Cold start under class loading, Flyway checks, connection pool warm-up and proxy creation can easily take **45 seconds** or more.

Recommended startup probe for a slow-starting payment path:

```
startupProbe:
  httpGet:
    path: /actuator/health/liveness
    port: 8080
  periodSeconds: 5
  failureThreshold: 12
  timeoutSeconds: 2
```

That gives roughly **60 seconds** before liveness is allowed to judge the pod, which is appropriate when the Spring context can take ~45 seconds under load.

13. High availability

Correct probe separation is an HA feature. A database outage is bad enough; it should not also erase every warm JVM by restarting them. Readiness preserves healthy process state while withdrawing traffic. That is exactly what you want during dependency instability and exactly what rolling updates depend on.

14. Disaster recovery

In a failover event, the fastest path back to service is often "pods stay up, then automatically rejoin when dependencies recover". That only happens when liveness is narrow and readiness is accurate. If both point at the same broad endpoint, disaster recovery becomes slower because the platform is busy recreating pods instead of waiting for the dependency to return.

15. Monitoring

Signal	Threshold	Why it matters
<code>kube_pod_status_ready{condition="false"}</code>	Rising during dependency event	Expected if readiness is working
<code>kube_pod_container_status_restarts_total</code>	Any rise during dependency event	Usually means liveness is wrong
Probe failure events	Repeated	Shows mis-tuning or genuine outage
Spring Boot health endpoint latency	Rising	Health checks themselves may be too expensive

16. Troubleshooting

Symptom	Cause	Command	Fix
Pod restarts during DB blip	Liveness tied to database	<code>kubectl get deploy -o yaml ; inspect probe paths</code>	Point liveness to <code>/actuator/health/liveness</code>
Pod never becomes ready	Readiness dependency unavailable or misclassified	<code>curl localhost:8080/actuator/health/readiness</code>	Fix dependency or health group
Probe endpoint 404	Probe groups not enabled	<code>curl localhost:8080/actuator/health/liveness</code>	Set <code>management.endpoint.health.probes.enabled=true</code>
CrashLoop during startup	No or too-short <code>startupProbe</code>	<code>kubectl describe pod</code>	Increase startup budget
Payments fail but pod stays ready	Readiness too shallow	Compare business errors with readiness state	Include critical dependency in readiness

Interview questions

1. Why should liveness and readiness not point at the same generic

`/actuator/health` **endpoint?** *Because they have different consequences. Readiness failure should stop traffic; liveness failure restarts the process. If both are tied to the same dependency-rich endpoint, a downstream outage causes unnecessary restarts.*

2. **What does `management.endpoint.health.probes.enabled=true` do?** *It enables Spring Boot's Kubernetes-oriented health groups so `/actuator/health/liveness` and `/actuator/health/readiness` are exposed with the intended semantics.*
 3. **Should liveness check a database? Why or why not?** *No. A slow or failed database does not mean restarting the JVM will help. It only destroys warm process state and turns a dependency fault into a restart storm.*
 4. **What belongs in readiness that does not belong in liveness?** *(senior) Critical downstream dependencies needed to serve the business operation, such as the database or Redis for shared fraud counters. If those are unavailable, the pod should leave rotation but continue running so it can recover quickly.*
 5. **How would you tune probes for a Spring Boot app that takes 45 seconds to start under load?** *(senior) Use a startup probe that grants at least 60 seconds of budget, then keep liveness stricter for steady-state failures and readiness faster for traffic draining. The startup probe prevents liveness from judging a pod that is still legitimately initialising.*
-

2.10 CPU Throttling Diagnosis for JVM Workloads

1. What it is

Understanding when a Java service is not "slow because it is busy" but slow because the Linux scheduler is repeatedly **stopping it from using CPU it momentarily needs**.

2. Why it exists

CPU limits are attractive because they look safe and tidy. For JVM workloads they are often the opposite: they punish short, intense bursts from GC, JIT compilation and request spikes even when the average CPU graph looks modest.

3. The business problem

`payment-service` had a CPU limit of `500m`. Average usage looked sensible: 220m to 300m. Nobody expected trouble.

Then Black Friday test traffic arrived. p99 latency climbed. GC pauses inflated. Thread pools backed up. Yet dashboards still showed average CPU below the limit most of the time.

The service was not starved on average. It was being **throttled in bursts**, exactly when garbage collection and request fan-out briefly needed more than half a core.

4. How it works

Linux CFS quota enforces CPU limits with a quota and period.

Setting	Meaning
<code>cpu.max = 50000 100000</code>	50 ms of CPU time per 100 ms period
Effective Kubernetes limit	<code>500m</code>

If the process consumes its 50 ms quota early in the 100 ms window, it is descheduled for the remainder. For a latency-sensitive JVM, that is long enough to matter.

```
100 ms period

allowed CPU time: 50 ms
JVM uses 50 ms quickly during GC or JIT
remaining 50 ms: throttled

average CPU may still look "fine"
latency does not
```

5. Internal architecture

Why JVMs are especially sensitive:

JVM activity	Why it bursts CPU
JIT compilation	Hot code is compiled in bursts by compiler threads
Garbage collection	Stop-the-world phases want CPU quickly
Request fan-out	Serialization, crypto and HTTP client work cluster together
ForkJoin / common pools	Parallel work assumes available processors are real

Since JDK 10, the JVM is container-aware and tries to detect available CPUs from cgroups. That helps, but it does not remove throttling; it only influences ergonomics such as GC and compiler thread counts.

Useful flags and concepts:

Setting	Purpose
<code>-XX:ActiveProcessorCount=<n></code>	Override detected processors when container CPU view is misleading
Container-aware defaults	JDK sizes internal pools based on cgroup CPU view
GC thread counts	Too many threads in a small quota intensify burst pressure

6. Component interactions

```
Deployment -> CPU request / limit
kubelet   -> writes cpu.max quota
JVM        -> detects available processors, sizes internal pools
GC / JIT   -> burst CPU briefly
kernel     -> throttles when quota exhausted
app        -> sees longer pauses and request latency
```

This is why "CPU under 50%" and "latency terrible" can coexist.

7. Enterprise example

Many banks use a policy split:

Workload type	CPU request	CPU limit
Customer-facing JVM API	Generous request	Often no CPU limit
Batch / async worker	Right-sized request	Limit acceptable
Shared node with many tenants	Moderate request	Limit may be necessary to contain neighbours

The policy is honest about the trade: removing CPU limits improves latency predictability but increases noisy-neighbour risk. There is no free lunch, only a conscious choice.

8. Real-world analogy

A cashier is allowed to work only 30 minutes of every hour, but customers arrive unevenly. Even if the day's average workload is light, a lunch rush still becomes a queue because the cashier is forcibly idle exactly when demand peaks.

Where it breaks: humans do not work in perfect 100 ms periods. CFS quota does, and that mechanical enforcement is why short bursts can be penalised so harshly.

9. Best practices

Practice	Reason
Set CPU requests from measured steady-state demand	Requests protect your share under contention
Be cautious with CPU limits on latency-sensitive Java services	Throttling is often worse than modest overuse
Measure throttling directly, not by guesswork	Average CPU hides burst starvation
Review GC and JIT behaviour when tuning limits	JVM internals create the bursts
Consider <code>-XX:ActiveProcessorCount</code> in constrained containers	Prevents wildly optimistic internal parallelism
Use load tests that reach p99 latency, not just average throughput	Throttling shows up in tails first
If you keep CPU limits, leave real burst headroom	500m on a synchronous API is often too tight

10. Common mistakes

Mistake	Symptom
Tight CPU limit with low average CPU	"Mystery" latency spikes
Looking only at mean CPU usage	Throttling missed entirely
Assuming more replicas always fix it	New JVMs start cold and may also be throttled
Ignoring GC pause inflation	Memory gets blamed for a CPU problem
Using CPU limits copied from non-JVM services	Same YAML, very different runtime behaviour
Noisy-neighbour fear leads to tiny limits everywhere	Platform looks controlled, user latency suffers

11. Security considerations

Threat	Control	Residual risk
One service monopolises shared node CPU	CPU requests, node isolation, optional limits	Without limits, abuse can still hurt neighbours
Deliberate CPU flood drives autoscaling cost	HPA caps, edge rate limiting	Attack can still consume reserved share
Overly permissive tuning flags expose internals	Standardise approved JVM options	Incident-time overrides may drift

12. Performance considerations

Signals that matter:

Signal	Source	Meaning
<code>container_cpu_cfs_throttled_periods_total</code>	Prometheus / cAdvisor	Count of periods where throttling happened
<code>container_cpu_cfs_throttled_seconds_total</code> or <code>throttled_usec</code>	Prometheus / <code>cpu.stat</code>	How much time was lost to throttling
<code>jvm_gc_pause_seconds</code>	JVM metrics	Pause inflation often follows throttling
<code>process_cpu_usage</code>	JVM view	Useful, but average-only by itself is insufficient

Inside the container:

```
kubectl exec -n axispay-core payment-service-6c7c7dbdb7-6m9r5 -- cat /sys/fs/cgroup/cpu.stat
```

Example output:

```
usage_usec 184930221
nr_periods 125640
nr_throttled 14982
throttled_usec 38700491
```

That tells you throttling is not hypothetical.

13. High availability

CPU throttling is an availability issue when it pushes latency beyond upstream timeouts. A service can remain `Ready`, pass every liveness check, never restart and still effectively disappear from the user path because callers give up. Silent latency failures are among the hardest availability failures to see.

14. Disaster recovery

During an incident, removing or raising a CPU limit is a legitimate break-glass action for a latency-sensitive JVM service. But it changes the cluster risk posture. The follow-up action is mandatory: if the service needed more CPU to survive, requests, quotas and node sizing must be revisited before the next peak.

15. Monitoring

Metric	Threshold
<code>container_cpu_cfs_throttled_periods_total</code> / <code>container_cpu_cfs_periods_total</code>	> 5% sustained
<code>container_cpu_usage_seconds_total</code>	Correlate with throttling, not alone
<code>jvm_gc_pause_seconds</code> p99	Sudden increase
API latency p99	Rising with throttling
HPA at max replicas plus throttling	Severe capacity / sizing problem

16. Troubleshooting

Worked example: `payment-service` limit `500m`, average CPU modest, latency terrible.

Observation	Meaning
Average CPU ~300m	Service is not continuously saturated
<code>nr_throttled</code> climbing fast	Short bursts are hitting the limit
GC pause p99 rising	GC cannot finish quickly when it needs CPU
More pods help only slowly	New JVMs take time to warm up

Diagnosis commands:

```
kubectl top pod -n axispay-core --containers | grep payment-service
kubectl exec -n axispay-core <pod> -- cat /sys/fs/cgroup/cpu.stat
kubectl describe hpa payment-service -n axispay-core
```

Possible fixes, in order of preference:

Fix	Benefit	Trade-off
Raise CPU request	Better guaranteed share under contention	Consumes schedulable capacity
Raise CPU limit	Reduces throttling	Still imposes a ceiling
Remove CPU limit	Best latency behaviour	More noisy-neighbour risk
Tune <code>ActiveProcessorCount</code> / thread pools	Smoother internal burstiness	More tuning complexity

The honest recommendation for latency-sensitive JVM services is often: **set memory limits, measure CPU carefully, and consider omitting CPU limits** unless your multi-tenant risk model demands them.

Interview questions

- How are CPU limits enforced in Linux containers?** *With CFS quota and period, exposed in cgroups as values such as `cpu.max`. A container gets a fixed amount of CPU time per period; once consumed, it is throttled until the next period.*
- Why are JVM workloads particularly prone to CPU throttling pain?** *Because GC, JIT compilation and request processing are bursty. They need short intervals of high CPU, and throttling those bursts inflates pause times and tail latency even when average CPU looks low.*
- What metrics or files prove throttling is happening?** *Prometheus metrics such as `container_cpu_cfs_throttled_periods_total` and cgroup data like `/sys/fs/cgroup/cpu.stat`, especially `nr_throttled` and `throttled_usec`.*
- Why can GC pause times rise while average CPU stays modest?** *(senior) Because GC needs burst CPU to finish quickly. If a 500m limit repeatedly stops it mid-burst, pause time stretches even though the long-term average CPU still appears acceptable.*
- When is removing a CPU limit the right decision?** *(senior) When latency predictability for a critical synchronous JVM service matters more than strict per-container CPU containment, and the platform has other controls such as generous requests, quotas, node isolation or trusted tenancy. It is a trade, not a default for every workload.*

2.11 Autoscaling Java Workloads: HPA Behaviour and Pitfalls

Java services interact awkwardly with the HPA because new capacity is not instantly useful. A Spring Boot pod may schedule in seconds and still take **30 to 90 seconds** before it contributes meaningful work: the JVM starts, classes load, connection pools warm, JIT compilation begins, caches are cold and readiness waits for dependencies. During that window, the HPA thinks it has added capacity, but the business path does not feel it yet.

That gap explains why default HPA behaviour often disappoints teams running `payment-service` or `fraud-service`. CPU rises, the HPA scales up, new pods appear, and latency still worsens for another minute. The loop is not "broken"; it is simply slower than the traffic event.

There is a second trap on the way back down. Default or aggressive scale-down can remove pods shortly after the spike passes, before the fleet has actually stabilised. For JVM workloads this is especially wasteful: the platform kills the very pods it just spent time warming up. If traffic oscillates, the service thrashes between cold scale-up and impatient scale-down.

The `behavior` field exists to shape that loop:

```
behavior:
  scaleUp:
    stabilizationWindowSeconds: 0
  scaleDown:
    stabilizationWindowSeconds: 300
```

For Java workloads, teams often go further and make the scaling loop intentionally less twitchy:

```
behavior:
  scaleUp:
    stabilizationWindowSeconds: 15
  scaleDown:
    stabilizationWindowSeconds: 600
```

That does two things. A short scale-up stabilisation window dampens single noisy samples without materially slowing response. A much longer scale-down window keeps warmed pods around long enough to absorb a second wave without paying another cold-start tax.

The metric choice matters even more than the timing. CPU is attractive because it is built in, but CPU is an imperfect proxy for payment demand. A flash-sale spike may produce queueing, request concurrency and downstream contention before mean CPU tells the full story. For `payment-service`, a custom metric such as in-flight requests (`http_server_requests_active`) or queue depth can be a better early signal than raw CPU, because it measures backlog or concurrency directly.

Worked example: an AxisPay merchant runs a Black Friday campaign at 09:00. `payment-service` starts with `minReplicas: 3`, `maxReplicas: 12`, CPU target 70%. Traffic jumps from 40 rps to 140 rps in under a minute.

What happens with a CPU-only HPA:

1. Existing pods saturate and CPU climbs.
2. HPA notices on the next metrics cycle and raises desired replicas.
3. New pods schedule quickly, but each needs ~60 seconds to become truly useful.
4. During that minute, the original pods remain overloaded and latency continues to climb.
5. By the time new pods help, the queue is already established.

To the business, "the HPA scaled" and "the service still struggled" feel contradictory. They are not. The autoscaler reacted correctly to the metric it had, but the new JVM capacity arrived too late to prevent the queue.

AxisPay's fix had two parts.

First, they increased `minReplicas` ahead of the event. Pre-warming is often the simplest and most effective solution for scheduled traffic. If the business already knows 09:00 will be exceptional, the platform should not wait for CPU graphs to discover it. Extra warm replicas cost money for a short window, but failed payments cost reputation immediately.

Second, they added a custom metric better aligned to user pain. Instead of waiting only for CPU, the HPA considered request concurrency. When `http_server_requests_active` rose sharply, scaling began earlier — before CPU averaged high enough to trigger the same response. CPU remained a useful secondary signal, but not the only one.

Example direction:

```
metrics:
  - type: Pods
    pods:
      metric:
        name: http_server_requests_active
      target:
        type: AverageValue
        averageValue: "30"
  - type: Resource
    resource:
      name: cpu
      target:
        type: Utilization
        averageUtilization: 70
```

This is not "custom metrics are always better". It is "scale on the bottleneck you actually hit". For a synchronous payment API, concurrency or queue depth often predicts pain earlier than mean CPU. For a CPU-bound signer or fraud scorer, CPU may still be the right primary metric.

Scale-down deserves equal care. If new pods take 60 seconds to become productive, killing them 90 seconds after the spike passes is perverse. A longer `behavior.scaleDown.stabilizationWindowSeconds` — often 300 to 600 seconds for JVM services

— keeps warmed capacity around long enough to ride out normal traffic aftershocks. The cost increase is deliberate and usually minor compared with the instability avoided.

Practical rules for Java HPAs:

Rule	Why
Keep <code>minReplicas</code> high enough for normal burst absorption	HPA is not instantaneous
Use <code>startupProbe</code> and readiness correctly	Unready pods are not useful capacity
Lengthen scale-down stabilisation	Protect warm pods from churn
Pre-warm ahead of known events	Scheduled business spikes are predictable
Consider custom metrics such as queue depth or active requests	They expose pressure sooner than CPU in many APIs

One final pitfall: an HPA can scale up perfectly and still fail operationally if the namespace quota, node capacity or downstream dependency cannot support the extra replicas. For Java workloads this hurts twice: you either get no new pods, or you get cold new pods that still cannot serve because the database or `merchant-service` is saturated. Autoscaling is not magic capacity creation. It is replica management within the limits of the rest of the system.

The sober conclusion is this: HPA works well for JVM services **when it is treated as part of a broader capacity design**. Warm baseline replicas, correct probes, realistic startup expectations, patient scale-down and the right metric together make autoscaling useful. Leaving the defaults untouched and hoping CPU alone will rescue a payment spike is how teams discover that "scaled up" and "handled the surge" are not the same sentence.

Day 2 cheat sheet

Resources

```
kubectl top nodes
kubectl top pods -n axispay-core --containers

# what is declared vs what QoS resulted
kubectl get pods -n axispay-core -o custom-columns=\
NAME:.metadata.name,\
CPU_REQ:.spec.containers[0].resources.requests.cpu,\
CPU_LIM:.spec.containers[0].resources.limits.cpu,\
QOS:.status.qosClass

# THE command for invisible latency - CPU throttling
kubectl exec -n axispay-core <pod> -- cat /sys/fs/cgroup/cpu.stat
#   nr_throttled / throttled_usec climbing = the limit is too low

kubectl describe quota -n axispay-core
kubectl describe limitrange -n axispay-core
```

Probes

Probe	Path	Fails →	Checks dependencies
startup	/startupz	keep waiting, liveness suspended	No
liveness	/healthz	RESTART	Never
readiness	/readyz	leave endpoints	Yes

```
kubectl get deploy <d> -n <ns> -o yaml | grep -A5 Probe
kubectl describe pod <pod> | grep -A3 "Liveness\|Readiness"
kubectl get endpointslice -n <ns> -l kubernetes.io/service-name=<svc>
```

Autoscaling

```
desired = ceil( current × utilisation ÷ target )      utilisation = usage ÷ REQUEST
```

```
kubectl get hpa -n axispay-core          # TARGETS <unknown> = no request
kubectl describe hpa <name> -n axispay-core | tail -15
```

Workload types

Need	Controller
N copies always running	Deployment
One per node	DaemonSet (no <code>replicas</code> field)
Run once and stop	Job (<code>restartPolicy</code> <code>Never/OnFailure</code>)
Run on a schedule	CronJob (set <code>timeZone</code> !)

```
kubectl get daemonset,job,cronjob -A
kubectl logs -n axispay-async job/recon-worker
kubectl create job --from=cronjob/settlement-cron manual-run -n axispay-async
```

Rollouts

```
kubectl rollout status deployment/<d> -n <ns>
kubectl rollout history deployment/<d> -n <ns>
kubectl rollout undo deployment/<d> -n <ns>
kubectl rollout undo deployment/<d> -n <ns> --to-revision=3

# measure it
kubectl port-forward -n axispay-ops deploy/loadgen 8090:8080 &
curl -s -X POST localhost:8090/api/v1/loadgen/start -H 'Content-Type: application/json' -d '{"rps":40}'
curl -s localhost:8090/api/v1/loadgen/stats | python3 -m json.tool
```

The two failure symptoms

	CPU limit exceeded	Memory limit exceeded
Result	Throttled	OOMKilled
Exit code	— (still running)	137
Restart	No	Yes
Log line	None	None from the app — SIGKILL cannot be caught
Event	None	Yes
Find it with	<code>cat /sys/fs/cgroup/cpu.stat</code>	<code>kubectl describe pod</code> → Last State

Day 2 review questions

1. Which does the scheduler use — requests or limits? Which does the kernel enforce?
2. What happens when a container exceeds its CPU limit? Its memory limit?
3. What are the three QoS classes, and in what order are they evicted?
4. Why does a ResourceQuota need a LimitRange?
5. Where does a quota rejection appear, and why is that easy to miss?
6. Name the three probes and, for each, the consequence of failure.
7. Why must a liveness probe never check a database?
8. How long after a pod becomes unready does traffic actually stop, and why is it not instant?
9. Give the HPA formula. What is the denominator?
10. An HPA shows `TARGETS: <unknown>`. What is wrong?
11. Why is scale-down slower than scale-up?
12. A dependency fails and payments error. What does the HPA do, and why?
13. Why does a DaemonSet have no `replicas` field?
14. What makes a Job complete? Why is `restartPolicy: Always` rejected?
15. Why must a CronJob set `timeZone`?
16. `replicas: 3`, `maxSurge: 1`, `maxUnavailable: 0` — max pods, min serving?
17. Describe the termination sequence from delete to SIGKILL.
18. Why does the Dockerfile `ENTRYPOINT` form affect graceful shutdown?
19. What are the two things that together make a rolling update zero-downtime?
20. `requests.memory: 96Mi`, `limits.memory: 48Mi`. What happens, and what should have caught it?

Answers: `documents/assessments/answer-keys/day2-answer-key.md`

Day 2 summary

You built: resource requests and limits on every workload · namespace governance verified to permit full autoscaling · three probes correctly separated by consequence · two HPAs scaling on real CPU load · a DaemonSet, a Job and a CronJob · a zero-downtime release of v1.1.0 under 40 rps with **zero failed payments**.

You proved: CPU throttling breaks an SLO with no log line · a wrong liveness probe turns a 40-second blip into a total outage · an HPA without requests does nothing and says nothing · removing the readiness probe drops real payments.

What is still missing:

Gap	Consequence	Fixed
No database	Every payment lost on restart. Nothing to reconcile.	L3.5
Config in manifests	A log-level change redeploys every service	L3.1
JWT key in a plain env var	Visible to anyone with namespace read access	L3.2
Fraud counters in memory	The control weakens as you scale — you found this yourself	L3.5
No persistent storage	A StatefulSet has nowhere to keep data	L3.3, L3.4
Containers run as root	A container escape becomes a host compromise	L3.7

Tonight (optional, 15 minutes): delete a `payment-service` pod, then try to fetch a payment you created this morning. Understanding *why* it is gone is the whole of Day 3.